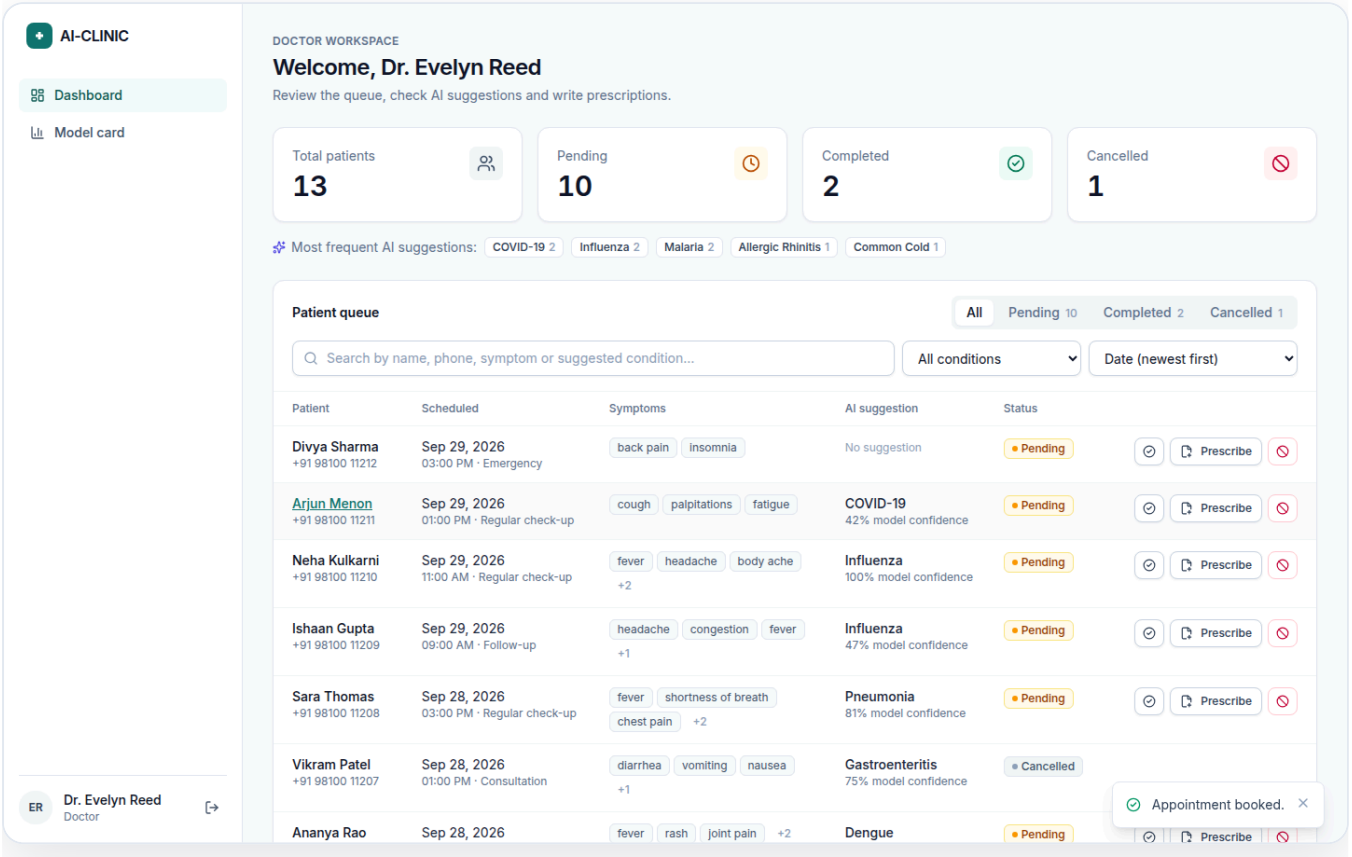
CODEBASE GUIDE & INTERVIEW PREP

AI-CLINIC

A short, file-by-file guide to the React + Flask + MongoDB + scikit-learn clinic app.

React 19 · Vite · Tailwind | TanStack Query | Flask 3 REST API | JWT | MongoDB Atlas | RandomForest | ReportLab PDF



github.com/Shoaib1M/AI-CLINIC

23 October 2026

1 The project in 60 seconds

> **ELEVATOR PITCH**
> "AI-CLINIC is a full-stack clinic workflow app with two roles. The **front desk** books appointments and records symptoms. A **scikit-learn random forest** ranks the top 3 likely conditions and stores that suggestion inside the appointment. The **doctor** reviews the queue, decides, and writes a prescription that is exported as a **PDF**. It's a **React** SPA calling a **Flask REST API** with **JWT** auth, backed by **MongoDB Atlas**."

Three things worth highlighting

1. **AI is decision support, not diagnosis.** Output is "model confidence" (tree agreement), always a top-3, styled differently (indigo vs teal), never auto-filled into a prescription.
2. **Honest evaluation.** The old 97.01% accuracy came from a bug (CSV header trained as a fake class "Disease") and duplicate rows leaking across a random split. Fixed with grouped cross-validation → **95.67% ± 2.33%**.
3. **Production-style engineering.** App factory, service layer, server-side validation, uniform JSON errors, role checks on the server, atomic ids, concurrency guards, graceful degradation if the DB or model is down, ~99 tests.

Numbers to remember

21 symptoms · **10** conditions	**200** trees	**1,000** rows · **338** distinct · **167** duplicates
95.67% ± 2.33% grouped CV	**100%** top-3 (hold-out)	**14** endpoints · **2** roles · **5** collections

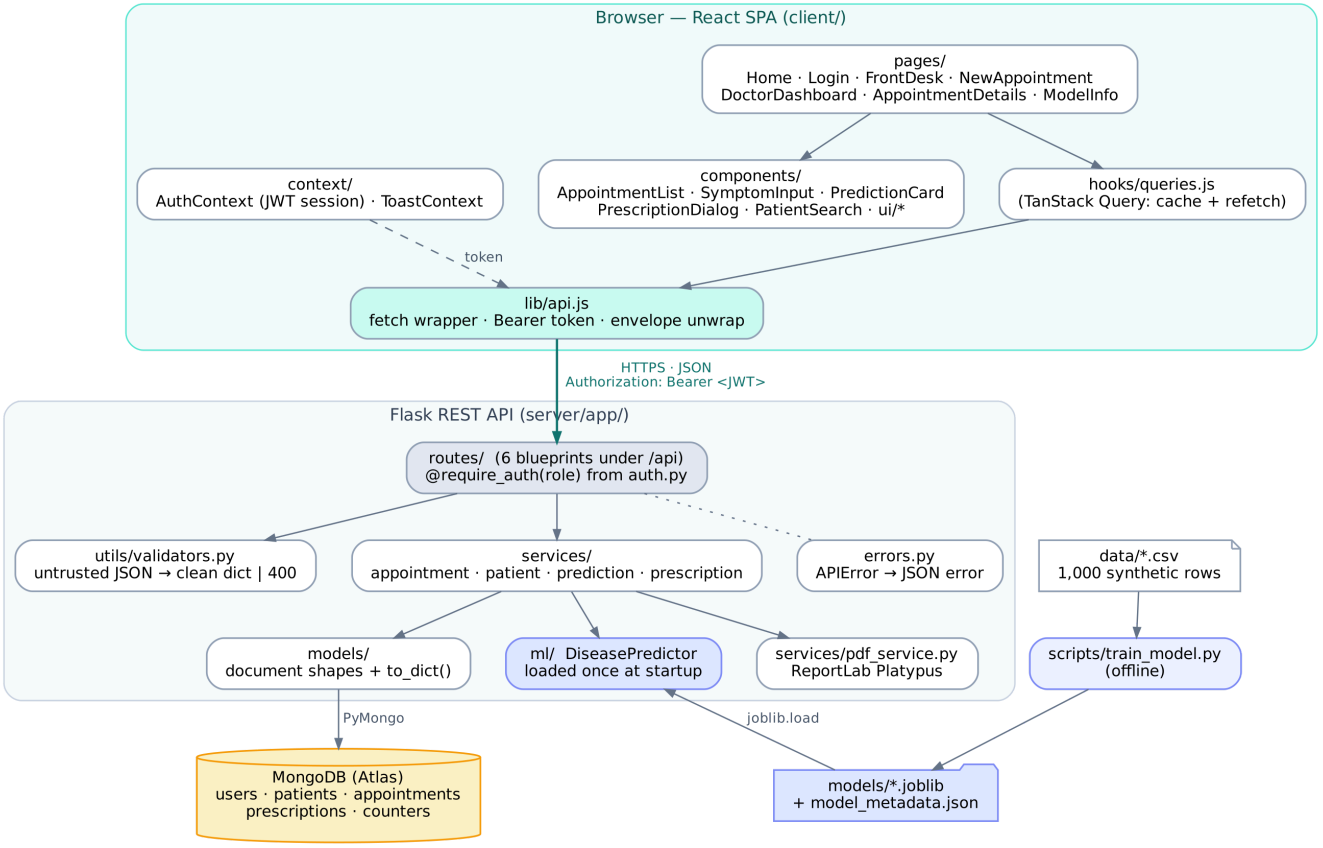
How it evolved

v1: single `app.py` with templates, `innerHTML` (XSS risk), hard-coded logins, model retrained on startup → v2: Flask REST API + SQLite + JWT, React frontend → v3 (now): **MongoDB Atlas**, because the data is document-shaped (symptom arrays, nested AI output, medication lists) and a managed DB keeps the API stateless.

Run it

```
cp .env.example .env        # set MONGODB_URI, JWT_SECRET, demo passwords
cd server && pip install -r requirements-dev.txt && flask --app wsgi seed-demo --with-appointments
flask --app wsgi run --debug                    # API :5000
cd ../client && npm install && npm run dev      # UI :5173 (proxies /api)
pytest  ·  npm test  ·  python -m scripts.train_model
```

2 Architecture



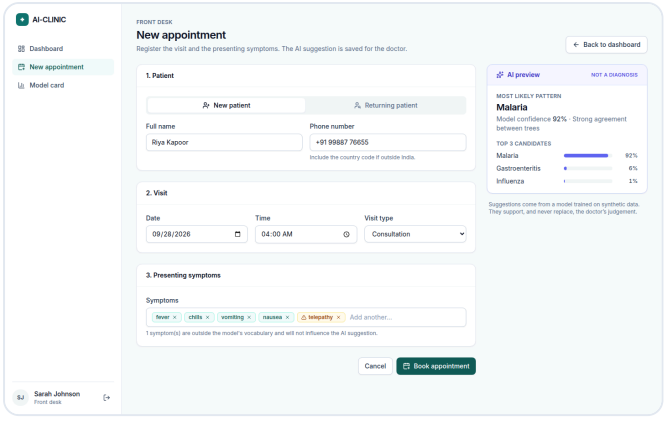
The React app only talks to the API via `lib/api.js`. Inside the API: routes → validators → services → MongoDB / ML / PDF. Training is a separate offline script.

> **SAY IT LIKE THIS**
>
> "Routes are thin: auth, validate, call a service. Services hold the rules. The ML package has no Flask or DB imports, so it's tested on its own. That separation let me swap SQLite for MongoDB without touching routes or the frontend."

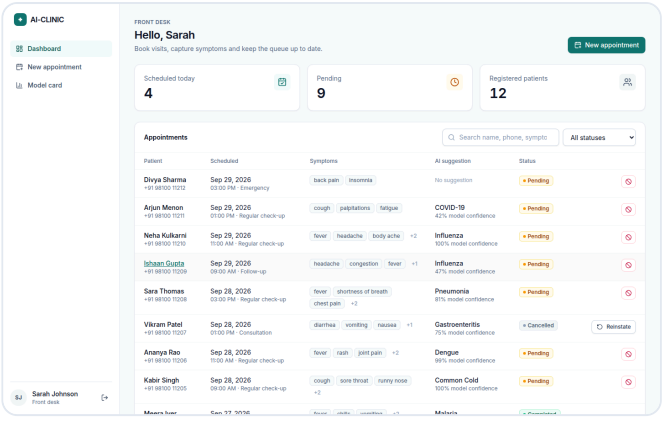
Booking flow (the one to narrate)

1. `NewAppointment.jsx` validates → `POST /api/appointments` with `Authorization: Bearer <JWT>`.
2. `@require_auth("frontdesk")` verifies the token, reloads the user from the DB, checks the role.
3. `validate_appointment_create()` returns a clean dict or `400` with **every** bad field.
4. Service finds/creates the patient, runs `prediction_record_for()` (never raises — booking never depends on the model).
5. One `insert_one` stores the appointment **with the prediction embedded** (atomic) → `201`.
6. TanStack Query invalidates `['appointments']` so both dashboards refresh.

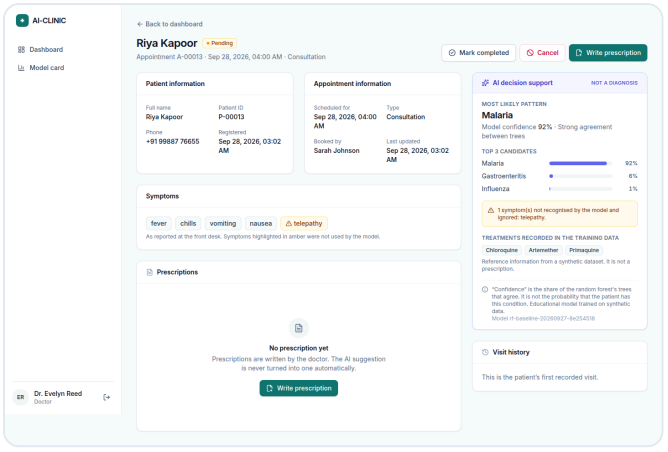
3 The app



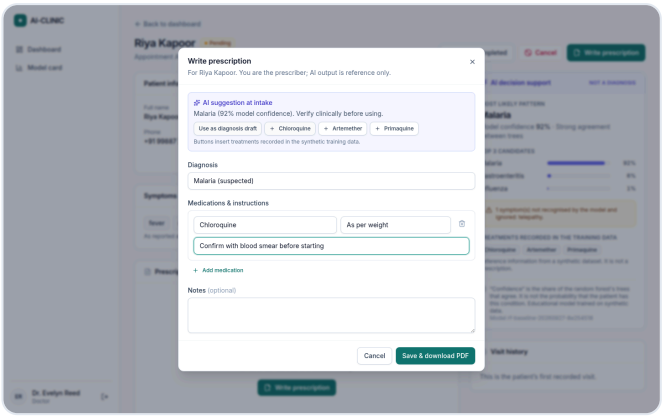
New appointment — symptom chips + live AI preview.



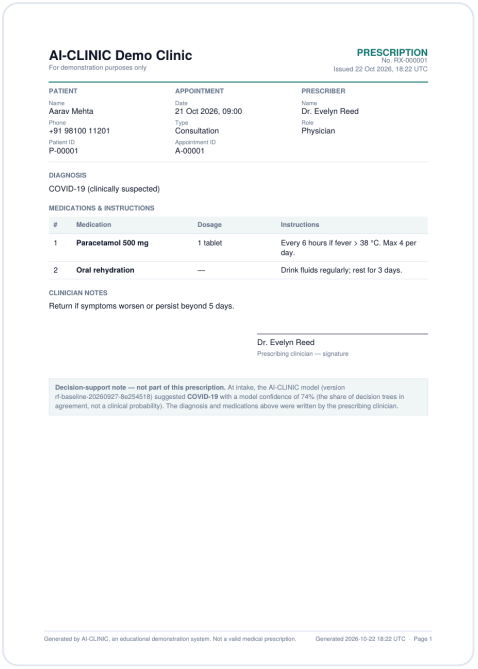
Front desk — stats, search, cancel/reinstate.



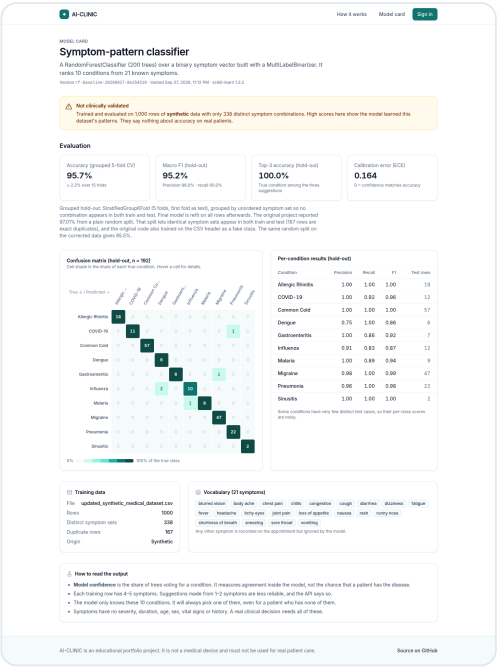
Details — "telepathy" is unknown, so it's ignored and flagged.



Prescription — starts empty; AI hints are click-to-insert.



Generated PDF — AI note is in a separate box.



Model card — metrics from `model_metadata.json`.

4 Backend — main files

`app/__init__.py – create_app()` CORE

App factory: loads config, logging, JWT secret check, Mongo client, CORS (only `/api/*`), error handlers, blueprints, CLI, security headers (`nosniff`, `DENY`, `no-store`). Creates indexes and loads the ML model **once**. If Atlas is unreachable it still starts and `/api/health` reports it. Tests build a fresh app per test.

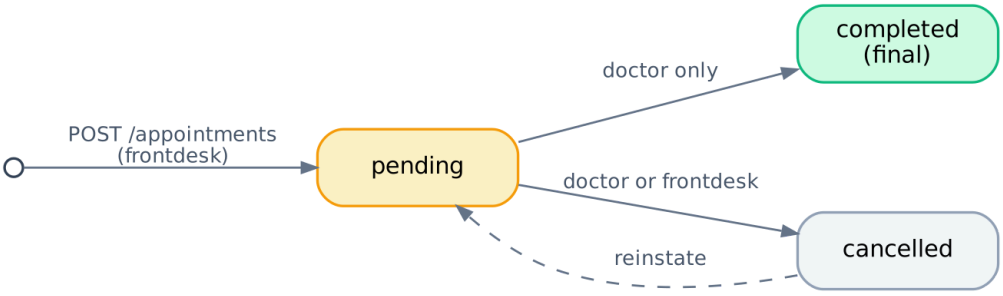
`app/auth.py` CORE

- `authenticate()`: same message and timing for "wrong password" and "unknown user" (checks a dummy hash) → no username enumeration. Passwords are salted hashes (Werkzeug).
- `issue_token()`: HS256 JWT with `sub, role, iat, exp` (8 h).
- `@require_auth(*roles)`: algorithm pinned (rejects `alg: none`), requires `exp/iat/sub`, **reloads the user from the DB every request** so deactivation/role changes apply instantly → 401 / 403.

`app/utils/validators.py` CORE

Untrusted JSON → cleaned dict or `ValidationError`. An `_Errors` collector gathers all field errors before raising, so the form can highlight them all at once. Highlights: phone 7–15 digits, Unicode-aware names, no past dates or timezone offsets, max 15 symptoms, PATCH rejects unknown fields (**no mass assignment**).

`app/services/appointment_service.py` CORE



- **State machine**: `TRANSITIONS` + `ROLE_STATUS_PERMISSIONS`; only doctors complete; completed is final → 403 / 409.
- **Concurrency guard**: `find_one_and_update({_id, status: old_status})` — if someone changed it first, nothing matches → `409 CONCURRENT_UPDATE`.
- **Search**: `re.escape()` before `$regex` (no regex injection). Sort uses a pipeline that keeps missing values last. Patients loaded in one `$in` query (no N+1).
- `stats()`: `$group` by status + top-5 predicted diseases.

`app/services/prediction_service.py` CORE

Loads the model at startup (never retrains). `predict()` powers the preview (`422` if no known symptoms). `prediction_record_for()` builds the stored sub-document with status `ok` / `no_known_symptoms` / `model_unavailable`.

app/services/prescription_service.py · pdf_service.py

CORE

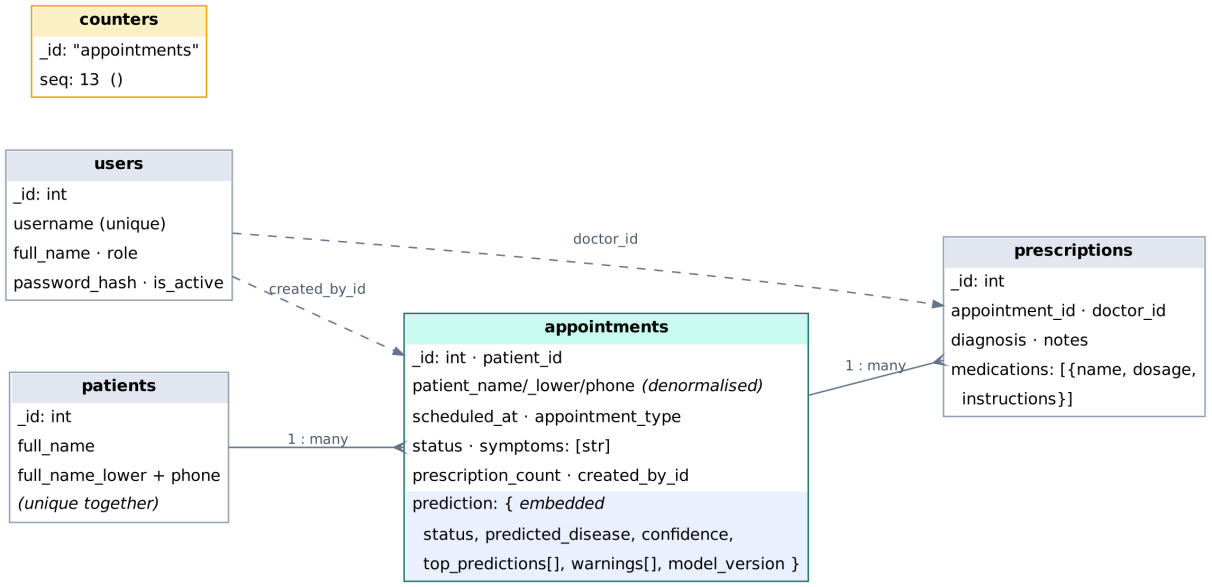
Doctor id comes from the **token**, never the body; no prescriptions for cancelled visits; `$inc prescription_count`. PDF uses ReportLab **Platypus** (auto-wrapping tables); every user string is XML-escaped so markup like `<font size=90>` prints literally.

app/extensions.py · errors.py · config.py

SUPPORT

`next_id()` gives atomic sequential ids via `$inc` on a counter doc. Unique indexes enforce invariants. `errors.py` maps an `APIError` hierarchy (400/401/403/404/409/422/503) to `{"error": {code,message,details}}`; unknown exceptions → generic 500. `config.py` has Dev/Testing (mongomock)/Prod; prod refuses a JWT secret under 32 chars.

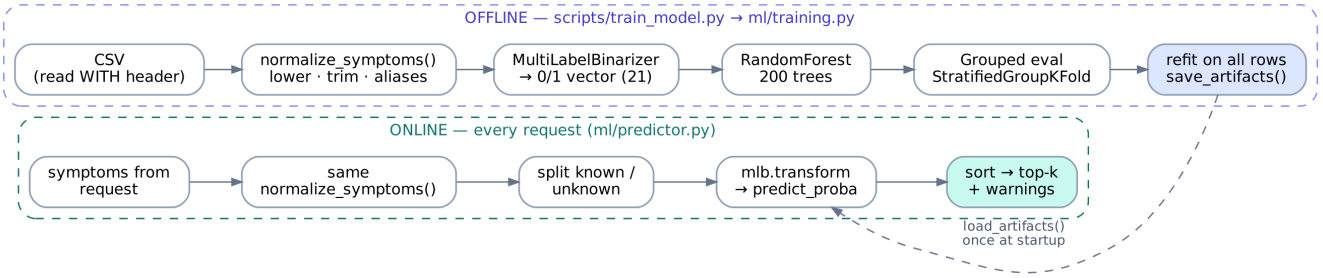
Data model



WHY EMBED THE PREDICTION BUT REFERENCE PRESCRIPTIONS?

Embed what is written once with its parent and always read with it (prediction → one atomic write, no join). Reference what has its own identity or grows (prescriptions have their own PDF URL and a visit can have several). Patient name/phone is copied onto appointments so the queue can search/sort in one collection.

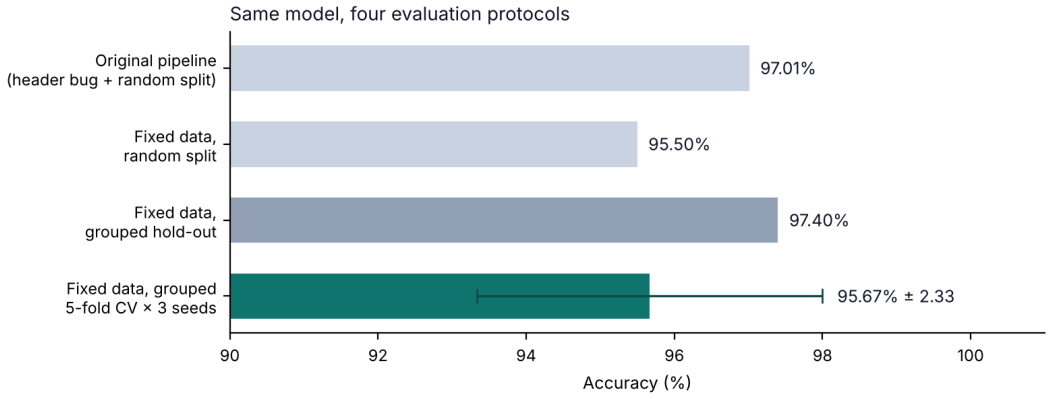
5 Machine learning



File	Role
`ml/preprocessing.py`	`normalize_symptoms()` : lower-case, trim, `_/-` → space, conservative aliases (`vomit` → `vomiting`). **Shared by training and serving** → no training/serving skew.
`ml/predictor.py`	`DiseasePredictor.predict()` : split known/unknown → `mlb.transform` → `predict_proba` → sorted top-k (deterministic ties) + warnings.
`ml/artifacts.py`	Save/load joblib files; refuses mismatched features/classes. sklearn pinned (pickles are version-bound).
`ml/training.py`	Load CSV **with header**, grouped hold-out + 5-fold × 3-seed `StratifiedGroupKFold` , refit on all rows, write metadata (version, dataset SHA-256, metrics).

How it works

MultiLabelBinarizer turns a symptom set into a 21-dim 0/1 vector. **Random forest**: 200 trees on bootstrap samples, √21 ≈ 4 features per split, Gini impurity. `predict_proba` ≈ share of trees voting for each class — shown as "model confidence", **not** a probability of disease (synthetic data, closed set of 10, uncalibrated: mean confidence 0.81 vs accuracy 0.97, ECE 0.164).



YOUR BEST ML STORY

"The reported 97% was wrong: the CSV header was trained as a fake class, and with only 338 distinct symptom sets, 76% of test rows had an exact copy in training. I fixed the loader and used grouped stratified CV, so a symptom set is never in both train and test: 95.7 ± 2.3. A tuned forest gained 0.6 points — inside the noise — with worse log loss, so I kept the simpler model."

5b Benchmarks — all numbers

Accuracy by evaluation protocol

Protocol	Accuracy
Original pipeline (header bug + random split, n=201)	97.01%
Fixed data, random split (duplicates leak)	95.50%
Fixed data, grouped hold-out (n=192)	97.40%
Grouped 5-fold CV × 3 seeds (15 folds)	**95.67% ± 2.33%**

Baseline vs tuned (grouped CV)

Config	Accuracy	Macro F1	Log loss
baseline (200 trees, deployed)	95.67% ± 2.33	93.40%	0.273
tuned (300 trees, leaf=2, balanced)	96.31% ± 1.65	94.18%	0.306

Grouped hold-out (808 train / 192 test)

Accuracy	97.40%
Precision (macro)	95.95%
Recall (macro)	94.96%
F1 (macro)	95.15%
F1 (weighted)	97.40%
Top-3 accuracy	100.00%
Log loss	0.246
ECE (calibration error)	0.164 (mean conf. 0.81 vs acc. 0.974)

Confidence on sparse vs full inputs

Input	Top 3 (model confidence)
fever	COVID-19 .18 · Influenza .18 · Malaria .18
fever, cough	COVID-19 .29 · Pneumonia .22 · Common Cold .14
fever, cough, fatigue, body ache, loss of appetite	COVID-19 .75 · Influenza .23 · Malaria .02
fever, chills, vomiting, nausea	Malaria .92 · Gastroenteritis .06 · Influenza .01

Per condition (hold-out) and class sizes in the full dataset

Condition	Precision	Recall	F1	Test rows	Dataset rows
Allergic Rhinitis	1.000	1.000	1.000	18	112
COVID-19	1.000	0.917	0.957	12	99
Common Cold	1.000	1.000	1.000	57	99
Dengue	0.750	1.000	0.857	6	124
Gastroenteritis	1.000	0.857	0.923	7	100
Influenza	0.909	0.833	0.870	12	101
Malaria	1.000	0.889	0.941	9	92
Migraine	0.979	1.000	0.990	47	92
Pneumonia	0.957	1.000	0.978	22	95
Sinusitis	1.000	1.000	1.000	2	86

Confusion matrix (hold-out, rows = true, columns = predicted)

True ↓ / Pred →	Allerg	COVID-	Common	Dengue	Gastro	Influe	Malari	Migrai	Pneumo	Sinusi
Allergic Rhinitis	18	0	0	0	0	0	0	0	0	0
COVID-19	0	11	0	0	0	0	0	0	1	0
Common Cold	0	0	57	0	0	0	0	0	0	0
Dengue	0	0	0	6	0	0	0	0	0	0
Gastroenteritis	0	0	0	0	6	0	0	1	0	0
Influenza	0	0	0	2	0	10	0	0	0	0
Malaria	0	0	0	0	0	1	8	0	0	0
Migraine	0	0	0	0	0	0	0	47	0	0
Pneumonia	0	0	0	0	0	0	0	0	22	0
Sinusitis	0	0	0	0	0	0	0	0	0	2

5/192 errors (overlapping symptoms). Tiny classes are noisy — quote CV. Data: 745 rows × 5 symptoms, 255 × 4; 6 label-noise sets; 76% random-split leakage.

6 Frontend — main files

File	What to know
`main.jsx · App.jsx`	Providers: Query › Router › Toast › Auth. Routes use layout routes + nested `ProtectedRoute roles=[…]` (UX only — the API enforces).
`lib/api.js`	Only place that calls `fetch`. Adds Bearer token (injected via `configureApi`), unwraps `{data, meta}`, throws `ApiError`, auto-logout on 401. PDF downloaded as a blob because a plain link can't send the header.
`context/AuthContext.jsx`	Session in `localStorage`; drops expired sessions; timer logs out at expiry; logout clears the query cache.
`hooks/queries.js`	All server state via TanStack Query. Hierarchical keys → invalidating `['appointments']` refreshes lists, details and stats. `keepPreviousData` for pagination.
`SymptomInput.jsx`	Accessible combobox chip input; autocomplete from model vocabulary; unknown symptoms shown in amber.
`PredictionCard.jsx`	Indigo "AI · not a diagnosis" card: top-3 bars, warnings, reference treatments.
`NewAppointment.jsx`	Debounced live preview with known symptoms only; maps server error keys back onto form fields.
`DoctorDashboard.jsx`	Filters live in the **URL** (shareable, back-button friendly); only the request is debounced.
`PrescriptionDialog.jsx`	Dynamic medication rows with stable keys; AI hints only on click; save → PDF download.

Supporting files: `AppointmentList` (table on desktop, cards on mobile), `ui/Modal` (native `<dialog>` for free focus trapping), `useStatusChange` (confirm dialog + mutation), `useDebouncedValue`, `Home`, `Login`, `ModelInfo`.

Tests: ~80 pytest (auth attacks, status rules, validation, ML, PDF escaping, DB outage — on in-memory mongomock) + 19 Vitest (API client, login/guards, symptom input, booking flow).

7 Weak spots & interview Q&A

FIX BEFORE YOUR INTERVIEW

`.env.example` is committed to GitHub and contains a **real MongoDB Atlas username and password**. Rotate that password in Atlas now and replace the value with a placeholder. An interviewer who opens the repo will spot it.

Known limitations (own them)

- JWT in `localStorage` (readable by injected scripts) — better: HttpOnly cookie + CSRF.
- No login rate limiting, refresh tokens or MFA.
- Patient + appointment, and prescription + `$inc`, are not wrapped in a transaction.
- Unanchored `$regex` search can't use indexes — fine for a demo, use a text/Atlas Search index at scale.
- Model: synthetic data, closed set of 10, no severity/age/vitals, not calibrated.

Likely questions

Q. Why Flask REST API + React instead of server-rendered pages?

> Clear contract testable without a browser, independent deployment (static site + API), and all rules in one place the client can't skip.

Q. Why MongoDB over SQL?

> Document-shaped data, managed Atlas keeps the API stateless, prediction fields can grow without migrations. Trade-off: no schema/foreign keys, so validators act as the schema and unique indexes + conditional updates protect invariants.

Q. How is authorisation enforced?

> On the server per endpoint with `@require_auth(role)`, using the role from the DB, plus per-action rules (frontdesk can cancel but not complete). Frontend guards are UX only.

Q. What if two users change the same appointment at once?

> The update filters on the status that was validated; the second one matches nothing and gets 409.

Q. What if the model file is missing?

> The API still starts; bookings succeed with `model_unavailable`, `/api/predictions` returns 503, health shows degraded.

Q. Why a random forest?

> Binary features, small data, non-linear symptom combinations, trains in seconds, vote share is explainable. Tuning didn't beat it beyond noise.

Q. What would you improve next?

> HttpOnly cookie auth + rate limiting, transactions, doctor assignment, recording the doctor's final diagnosis as a feedback loop, real de-identified data with calibration, Docker + CI.